IMPERIAL

MEng Individual Project

Imperial College London

Department of Computing

LLM-Driven Seed Corpus Synthesis for Fuzzing

Project Report

Supervisor:
Prof. Cristian Cadar

Author:
Fares Shehata

Assistant Supervisor:
Bachir Bendrissou

Second Marker:
Prof. Alastair Donaldson

June 9, 2026

Contents

Chapter 1

Introduction

Testing software is important. Across the U.S. economy, the cost of poor software quality was estimated at at least $2.41 trillion in 2022, with technical debt around $1.52 trillion - illustrating the economic risk of defects and the value of rigorous verification and validation [1, 2]. However, maintaining a high-quality test suite is challenging: it is difficult to enumerate all relevant inputs, states, and corner cases, and infeasible to exercise every potential path through complex software systems. Systematic techniques like fuzzing help by exploring vast input spaces - including malformed or unexpected inputs that traditional test design may miss - and can be run continuously with runtime feedback (e.g., sanitizers, coverage) to catch regressions early [3, 4].

Fuzzing has proven to be a very successful technique to find bugs and vulnerabilities in code. Some of the most important software we use - language compilers, operating system components, cryptography libraries, and database systems - have had critical bugs discovered and fixed via fuzzing [3]. For example, Google's large-scale fuzzing infrastructure has reported thousands of security vulnerabilities and tens of thousands of functional bugs across hundreds of open-source projects, leveraging coverage-guided engines (e.g., libFuzzer, AFL++) with sanitizers and scalable orchestration [5]. These systems rely on code-coverage guidance to steer execution toward new edges and behaviours; while coverage cannot literally guarantee "all code paths" are explored, it provides a practical, measurable heuristic to maximize path diversity during a campaign [5, 6].

There are two major categories of fuzzers: generation-based and mutation-based. Generation-based fuzzers use input formats or grammars to synthesize inputs from scratch that satisfy syntactic rules of the SUT. Mutation-based fuzzers begin with a corpus of example inputs (seeds) and iteratively mutate them [3]. In practice, grey-box mutation-based fuzzers often dominate modern vulnerability discovery because they (a) start from real inputs that already respect many syntactic/semantic constraints, enabling rapid progress past shallow checks, and (b) use lightweight program feedback (e.g., edge coverage, value profiling, branch distances) to adapt mutations toward novel paths and deeper states - especially effective for uncovering unexpected or undefined behaviour in complex code bases [3, 7].

Despite the strengths of mutation-based fuzzers, their effectiveness often hinges on the quality and variety of seed inputs. A narrow or unrepresentative corpus can trap a campaign in local minima, limiting path diversity and bug discovery; conversely, a rich, diverse seed set boosts pass rates through parsers and unlocks deeper program states [4]. Manually crafting such corpora is labour-intensive and time-consuming, and using existing samples "from the wild" offers no guarantee of exercising the full behavioural envelope of the SUT. As a result, many fuzzers invest in seed curation, deduplication, minimization, and dynamic corpus management to keep exploration efficient and targeted [7].

With the advent and widespread adoption of large language models (LLMs), their reasoning and contextual knowledge can alleviate several setbacks of existing fuzzing approaches. In mainstream software assurance, early evidence from large-scale industrial deployments shows that AI assistance can expand test generation and accelerate the on-boarding of new targets to continuous fuzzing; for example, Google reports that AI-generated fuzz targets and helpers have led to more vulnerabilities

being found, and expanded coverage across diverse ecosystems [6, 8]. In fuzzing research, LLMs have been used to generate grammars and structured inputs, to mutate inputs with semantic awareness, and to broaden language and domain coverage - spanning compilers, APIs, protocol/file formats, and more [9, 10, 11, 7].

Work has been done in using LLMs to generate fuzzers themselves, guided by coverage data [11], as well as generating inputs directly with a simple crash oracle [9, 12]. However, we have not seen any projects using LLMs to generate inputs, but also in a coverage guided manner. This is what our project aims to explore, as good quality seeds are essential to the effectiveness of mutation-based fuzzers, and using coverage information to curate those seeds could result in a great increase in fuzzing ability.

Chapter 2

Background

2.1 Fuzzing

Fuzzing is an automated software testing technique that exercises a system under test (SUT) with large numbers of inputs - often randomly generated or systematically mutated - and observes the SUT for failures or other "interesting" behaviours [13, 14]. Historically, seminal studies by Miller and colleagues demonstrated that random inputs could reliably uncover robustness issues in UNIX utilities and services, popularizing the approach and coining the term "fuzz" [13, 14]. At minimum, a fuzzing campaign requires (i) a mechanism to obtain test inputs (e.g., via generation or mutation), and (ii) an oracle to judge whether an execution is interesting (e.g., crashes, sanitizer-detected undefined behaviour, coverage increases, or differential inconsistencies) [15, 16, 17, 18, 19, 20]. The term "fuzzing" encompasses multiple sub-categories: generation-based vs. mutation-based input creation, and black-box vs. grey-box vs. white-box feedback. These categories trade off generality, required access to the SUT, and effectiveness [14, 15].

Oracles can be defined in several ways. Classical "crash oracles" detect fatal signals or abnormal exits. Sanitizer oracles detect memory and concurrency errors at runtime - for example, AddressSanitizer (ASan) for buffer overflows and use-after-free [16], MemorySanitizer (MSan) for uses of uninitialized memory [17], UndefinedBehaviorSanitizer (UBSan) for undefined language behaviors [18], and ThreadSanitizer (TSan) for data races [19]. Coverage change can also serve as a meta-oracle to prioritize inputs that explore previously unseen code [15, 21]. Finally, differential testing oracles flag behavioural inconsistencies across multiple implementations or compiler/toolchain variants (e.g., Csmith-based cross-compiler testing, or Equivalence Modulo Inputs) [22, 20].

2.1.1 Generation-based fuzzing

Generation-based fuzzers synthesize inputs from scratch, often guided by formal grammars or input specifications, to produce syntactically valid (and sometimes semantically constrained) test cases [23, 24]. This approach is powerful when a grammar or model is available and correctness of the input structure is important. Representative systems include:

- **Csmith**, which generates well-defined C programs to differentially test compilers while avoiding undefined behaviour [22];
- **LangFuzz**, which composes code fragments guided by a grammar to stress language processors [24];
- **NAUTILUS**, which couples grammars with coverage guidance to explore deep states in structured input spaces [23].

A known challenge for generation-based fuzzing is probabilistic bias: the distribution over grammar productions (or construction rules) influences which features and paths are explored and which bugs are likely to surface. Certain early failures (e.g., parser rejects) may mask deeper behaviours. Design choices such as avoiding undefined constructs (as in Csmith) and integrating feedback into generation (as in NAUTILUS) mitigate these issues [22, 23]. Another limitation is domain

specificity: high-quality generators require grammar/specification knowledge and are not trivially reusable across unrelated SUTs [24].

2.1.2 Mutation-based fuzzing

Mutation-based fuzzers derive new inputs by mutating existing seeds, leveraging that real inputs satisfy many syntactic/semantic constraints and thus can bypass shallow checks [15]. Mutations range from low-level bit/byte flips and arithmetic on integers, to structural operators such as block insertion/deletion/duplication, splicing segments from multiple seeds, and dictionary/token-aware rewrites [21]. For example, in the AFL/AFL++ family of grey-box mutation fuzzers, the fuzzing loop is feedback-directed and proceeds roughly as follows:

1. Initialize a queue (corpus) from seed inputs.
2. Select a seed according to a power schedule that favours seeds with higher novelty/coverage potential, allocating them more mutation "energy" [15].
3. Apply a sequence of mutation stages, including deterministic bit/byte flips and small arithmetic tweaks, followed by randomized "havoc" (stacked mutations) and optional splicing with other seeds.
4. Execute the mutated input on an instrumented target using a low-overhead execution model (e.g., forkserver or persistent/in-process execution) to maximize throughput, with instrumentation added at compile time or via binary-level dynamic translation when source is unavailable [15, 25, 26, 21].
5. Record the coverage bitmap (e.g., edge coverage with hitcounts) and, if the input exercises previously unseen edges or yields a sufficiently "interesting" coverage change, enqueue it as a new seed and update scheduling statistics.
6. Triage, deduplicate (e.g., by coverage/signature), and minimize crashing inputs for efficient regression testing.

AFL++ extends this loop with numerous engineering improvements (e.g., richer mutation operators and scheduling heuristics, dictionaries, context-sensitive coverage, and persistent execution modes) while retaining the same core principle: lightweight feedback steers mutations toward inputs that explore novel paths and uncover deeper bugs [15, 21].

While highly effective, mutation-based fuzzing is sensitive to seed quality and diversity: a narrow seed set can trap the search in local minima, whereas diverse seeds improve parser pass rates and enable deeper exploration. Consequently, corpus management (seed curation, minimization/deduplication, and power scheduling) is a first-class concern in production fuzzers [15, 21].

2.1.3 Black-box, grey-box, and white-box feedback

Feedback-directed fuzzing adapts input generation/mutation based on information obtained from the SUT [15]. The extent and type of information distinguishes three broad modes:

- **Black-box** fuzzing executes the unmodified SUT and observes only external outcomes (exit codes, crashes, timing). It suits closed-source targets and can maximize throughput but offers limited guidance. Differential testing of compilers with Csmith is largely black-box with respect to compiler internals [22].
- **Grey-box** fuzzing uses lightweight instrumentation to obtain coarse-grained internal signals - most commonly edge or branch coverage, value profiling, and sanitizer findings - balancing guidance and throughput. Instrumentation can be added at compile time (e.g., compiler-based edge counters and sanitizer hooks, as in libFuzzer/Clang) or at the binary level via dynamic instrumentation (e.g., Pin) or dynamic translation/emulation (e.g., QEMU), with some performance cost [21, 26, 25].
- **White-box** techniques analyse code paths symbolically and use constraint solvers to generate inputs that satisfy path predicates; these are often considered symbolic or concolic execution rather than fuzzing per se. Classical systems include DART (directed automated random

testing), KLEE (systematic test generation for systems software), and SAGE (white-box fuzzing at scale) [27, 28, 29]. White-box approaches can be highly precise but often incur significant computational cost to manage path explosion and constraint solving.

2.1.4 Instrumentation, coverage, and sanitizers as oracles

Coverage instrumentation records which code elements are executed - line/statement coverage, branch coverage, edge coverage, and function coverage - providing both progress metrics and guidance for feedback-directed search [30]. In grey-box fuzzing, edge coverage (control-flow-graph edge hit maps) has become a de facto signal because it captures path diversity at low overhead. Compiler toolchains expose coverage counters (e.g., via sanitizer/coverage passes), and binary-level tools can approximate similar metrics [15, 26, 25]. Inputs that exercise previously unseen edges, or significantly alter the coverage profile, are prioritised for further mutation [15, 21].

Sanitizers enrich oracles by detecting classes of undefined or erroneous behaviour at runtime, also with compile time instrumentation.

- **AddressSanitizer (ASan)** detects invalid memory accesses (buffer overflows, use-after-free) with relatively low overhead. [16].
- **MemorySanitizer (MSan)** detects uses of uninitialized memory [17].
- **UndefinedBehaviorSanitizer (UBSan)** detects undefined behaviours such as signed integer overflows and invalid shifts [18].
- **ThreadSanitizer (TSan)** detects data races in multithreaded programs [19].

Collectively, coverage and sanitizers serve both as guidance signals and as oracles of "interestingness," enabling fuzzers to triage and minimize crashing or sanitizer-violating inputs [15, 21].

2.2 LLMs

Large language models (LLMs) are neural networks trained to predict sequences of tokens and thereby generate, understand, and reason about text and code. The decisive architectural breakthrough was the Transformer, which replaced recurrence with self-attention, enabling efficient parallel training and better modelling of long-range dependencies [31]. From this foundation, families of models diversified: encoder-only models for representation learning (e.g., BERT) [32]; decoder-only, autoregressive models that excel at in-context learning and few-shot generalisation (e.g., GPT-3) [33]; and, at larger scales, systems such as PaLM that demonstrated gains from the triumvirate of data, parameters, and compute [34]. Alignment advances, notably instruction tuning and reinforcement learning from human feedback (RLHF), further improved models' ability to follow natural instructions safely and reliably [35]. Open foundation models (e.g., LLaMA) broadened access and accelerated downstream innovation [36], while code-specialised LLMs (e.g., StarCoder and Code Llama) brought stronger capabilities to software engineering tasks [37, 38].

In software testing, LLMs have been adopted to reduce the cost of writing target-specific generators and harnesses, and to expand test diversity across languages and evolving features. For compilers and language processors, LLMs can automatically produce diverse, valid programmes that increase coverage and expose nuanced feature interactions [9, 39]. White-box flows integrate LLMs with compiler analyses to derive targeted test cases for particular optimisation passes [10]. Beyond test synthesis, LLMs help generate fuzz drivers and adapt prompts from documentation, cutting set-up time for continuous fuzzing [40, 41]. In lower-level domains, code-aware LLMs have been used to exercise deep kernel execution paths and uncover subtle defects [42]. Empirically, universal LLM fuzzers demonstrate significant coverage gains and numerous bug findings across compilers, SMT solvers, and quantum stacks, illustrating the breadth and productivity benefits of LLM-assisted testing [9, 12].

Nonetheless, naively sampling large models can yield low validity - especially in strict languages - and waste SUT cycles. Practical systems therefore combine LLM generation with principled loops that distil prompts, adapt generation strategies, and repair errors on the fly to raise the proportion of valid tests and sustain exploration [9, 12]. Looking ahead, hybrid designs that pair selective

feedback signals (e.g., simple coverage counters or domain oracles) with LLM breadth are likely to deepen path exploration further, while maintaining portability across languages and SUTs.

2.3 Literature Review

2.3.1 ELFuzz

ELFuzz proposes a new method of LLM-based fuzzing: instead of directly generating test inputs, it uses an LLM to synthesize generation-based fuzzers themselves, and then evolves those fuzzers using a coverage-guided notion of "fuzzer space", until they become strong seed generators for downstream mutation fuzzers (e.g., AFL++). This decomposition turns the hard problem of writing a good generator into many small, LLM-friendly code edits with quantitative feedback at each step [11].

The motivation behind this approach is that building a high-quality generation-based fuzzer is difficult, as human-written grammars and semantic constraints are hard to author and maintain at scale. By generating and evolving a collection of fuzzers with an LLM, and ranking them by the coverage they achieve, the burden of manual specification is shifted to the model while progress is governed by an objective measure.

How ELFuzz works

ELFuzz implements an iterative three-stage evolution loop; each iteration "climbs" toward stronger fuzzers (Figure 2 and Figure 3 in the paper):

(1) LLM-driven mutation

Given a current pool of candidate fuzzers (small programs that emit inputs), ELFuzz asks the LLM to produce code-level mutants via three human-like edit operations:

- Splicing: combine head of one fuzzer with tail of another using fill-in-the-middle to glue the junction. This merges strengths of two candidates in one mutant.
- Completion: truncate a candidate at a random point and let the LLM complete the remainder, often adding missing logic and utility calls.
- Infilling: remove internal lines ("holes") and let the LLM fill them while maintaining consistency with surrounding code.

These operations leverage modern LLM editing capabilities to produce syntactically valid, semantically meaningful variants that can plausibly generate more diverse and valid inputs than naive random code edits..

Ablation results show that removing the fuzzer-space model hurts the most, and among mutators, removing splicing causes the largest degradation (up to 26.2% on several benchmarks), consistent with its role in fusing complementary strengths.

(2) Fuzzer space exploration (coverage-based strength analysis)

Each mutant (and existing seed fuzzer) is executed briefly to approximate the "cover set" of code elements (edges) it can reach in the SUT.

Let $C(F)$ and $S(F)$ denote the cover set and strength of a fuzzer F respectively. ELFuzz defines a strength pre-order:

$$S(F_i) \sqsubset S(F_j) \quad \iff \quad C(F_i) \subset C(F_j)$$

Mutants strictly weaker than their seed parents (e.g., due to type errors or reduced coverage) are discarded; those that introduce new coverage are retained. Because different fuzzers can cover disjoint regions, some are incomparable (neither subset nor superset). This induces a lattice structure - the "fuzzer space" - over candidate generators. Crucially, it enables fine-grained comparisons that a single scalar (e.g., total coverage count) would obscure.

(3) Max-cover selection (elite picking to prevent pool blow-up)

To keep the population small and diverse, ELFuzz selects a fixed number (N) of elites whose union of cover sets is maximized. Let V be a pool of fuzzers, and $E \subset V$ be the subset which maximises the union of cover sets. $\mathcal{P}_N(V)$ represents all N-element subsets of V. Thus, we have:

$$E = \underset{S \in \mathcal{P}_N(V)}{\operatorname{argmax}} \left| \bigcup \{\mathcal{C}(F) | F \in S\} \right|$$

A greedy approximation (with re-seeding heuristics) is used to avoid an exponential search, while still favouring elites that collectively expand coverage.

Implementation notes:

The design is LLM-agnostic. The paper uses a local CodeLlama-13B model for cost reasons, with short prompts that state the SUT and format hints in comments. The entire evolution controller and fuzzer-space analysis are lightweight Python (3.6 KLoC) compared to classic grammar pipelines (e.g., 80 KLoC for Csmith) [22, 11].

Coverage is approximated by executing a limited number of generated tests per candidate and recording edge coverage; this "quick probe" is sufficient to rank candidates and guide search while keeping the loop fast [11, 30].

Benchmarks and reported results

Subjects under test (seven real-world SUTs, up to 1.79 MLoC) include structured formats and interpreters: jsoncpp (JSON), libxml2 (XML), re2 (RegEx), CPython (Python), cvc5 (SMT-LIB2), SQLite (SQL), and librsvg (SVG). Baselines include grammar-based and constraint-based input generators (e.g., GLADE/ANTLR pipelines and ISLA constraints) translated or adapted as needed for fair comparison.

Input-generation effectiveness (RQ1):

ELFuzz's synthesised fuzzers generate seed corpora with much higher coverage within 10 minutes; the paper reports up to 434.8% more covered edges than the second-best technique on cvc5, and consistent advantages across the other SUTs (Figures 7 and 8).

Bug-finding with AFL++ (RQ2):

Using ELFuzz seeds, AFL++ triggers more bugs and reaches milestones faster. On some targets (e.g., SQLite), ELFuzz seeds lead to up to 216.7% more injected bugs found within 24 hours, and lower time-to-25/50/75% of total bugs compared to the next best baseline (Table 6; Figure 9).

Real-world campaign (RQ2, cvc5):

Over 14 days with 30 AFL++ instances, the ELFuzz-seeded campaign found five previously unknown bugs; three were exploitable. Some were fixed upstream during the study (Appendix C).

Ablation study (RQ3):

Removing the fuzzer-space model (ELFuzz-NoFS) drops performance the most (gaps from 6.6% to 62.5% vs. the full system). Among mutators, removing splicing (ELFuzz-NoSP) yields the largest additional drop (up to 26.2%), while removing completion or infilling usually has smaller effects—consistent with splicing's role in combining complementary behaviours.

Interpretable/extensible generators (RQ4):

Case studies on SQLite show synthesised fuzzers encode non-trivial features (e.g., command-line pragma handling, query sorting) and can be extended by external techniques such as Zest-style coverage-guided parameter evolution (Appendix E).

Cost and practicality:

Synthesis time is a one-off pre-fuzzing cost (hours, varying by SUT), after which the synthesised generators are fast to run. The paper discusses a trade-off: heavy synthesis can reduce "wall-clock" available for subsequent mutation fuzzing, but the improved seeds often outweigh this cost in coverage and bug-finding speed.

Limitations mentioned in the paper

- The prototype initializes from very simple random-byte seed generators for generality. Using stronger initial templates (or LLM-discovered protocol/grammar hints) could further improve convergence.
- If an SUT's input language is poorly represented in the LLM's training data (e.g., proprietary/binary formats), the LLM's edits may lack the priors needed to synthesize good fuzzers. Prompt engineering or fine-tuning may be required, and purely binary formats remain challenging.
- ELFuzz currently optimizes the input-generation stage that seeds AFL++. Maintaining diversity continually throughout the entire mutation campaign (dynamic co-evolution of seeds) is not explored and is a direction for future work.
- On some targets, ELFuzz spends tens of minutes synthesizing seeds before the AFL++ phase; while seeds speed up bug discovery thereafter, the up-front cost can matter under tight time budgets.
- Results were achieved with a small local 13B parameter model. Larger models or tool-augmented prompting could improve edits, but at higher compute cost.

Takeaways and implications for our project

- The concept of "fuzzer space" provides an objective and quantifiable way to improve a set of generators. While this project will be working on seeds directly, using fuzzer space can also allow us to objectively quantify the strength of an input / set of inputs.
- Human-like code edits (splicing, completion, infilling) utilise the capabilities of the LLM to make intelligent and context-aware changes to generators. Again, we can apply this directly to seeds.
- Possible extensions mentioned in the paper that we could implement include: integrating semantic constraints when available (e.g., via constraint solvers or specs), running continuous co-evolution during the mutation phase, and exploring retrieval or fine-tuning for low-resource or proprietary domains.

2.3.2 Fuzz4All

Fuzz4All is a universal language fuzzing tool that aims to target a long-standing pain point in fuzzing: writing SUT-specific generators and mutation logic are brittle, labour-intensive, and hard to reuse across languages or fast-evolving features. Its core insight is to leverage two LLMs and an "autoprompting" stage to produce concise, task-specific prompts, then run an iterative fuzzing loop that uses examples and natural-language "generation strategies" to continuously create valid, diverse test programs and inputs for many SUTs and languages, without instrumenting the SUT [9].

How Fuzz4All works

Fuzz4All makes use of two LLMs, each for a different role:

- The **Distillation LLM** reduces verbose user inputs (language documentation or specification/examples) into an "input prompt" that succinctly captures what to fuzz and how (e.g., feature summaries, allowed forms, example shapes). In the paper, the authors use GPT-4 as the distillation LLM, as it is a large model tuned for complex instructional prompts. This is called the "autoprompting" phase

- The **Generation LLM** creates fuzzing inputs (programs, expressions, API calls, test cases) from the distilled prompt, combining natural language instructions with code snippets. Here they use the small local LLM StarCoder for generation, to reduce the inference cost associated with querying larger LLMs.

Autoprompting (distillation stage)

Given user inputs (target docs, feature descriptions, examples), the autoprompting algorithm produces and scores candidate prompts, returning the best one to start fuzzing.

Candidate generation:

Generating candidate prompts is split into two parts:

- First, a "greedy" prompt is used, querying the LLM with a temperature value of 0. This is tuned for obtaining a plausible prompt with high confidence.
- Then, sample several variants at higher temperature are queried to diversify phrasing / structure while staying faithful to the task.

Scoring:

Each candidate prompt is used to drive a short generation run, and a scoring function evaluates the prompt by the validity of its produced code with respect to the SUT's acceptance predicate (e.g., "does the compiler accept this program?"). This is chosen as opposed to other possible metrics (e.g., coverage, bugs found) as valid inputs are most likely to exercise logic deep within the SUT, instead of just the frontend / input parsing phase.

Let p be a prompt, $\mathcal{M}_{\mathcal{G}}$ be the generation LLM, c be a code snippet, and isValid be a function that returns 1 if the code snippet is valid with respect to the SUT The scoring function is defined as:

$$\text{score}(p) \;=\; \sum_{c \in \mathcal{M}_{\mathcal{G}}(p)} \text{isValid}(c, \text{SUT})$$

The prompt with the highest score is chosen to pass on the fuzzing loop stage.

Fuzzing loop (generation stage)

Fuzz4All's loop iteratively updates the prompt and generates new inputs using three generation strategies, while a user-defined oracle flags bugs/crashes. A rolling "example" is stored in between iterations, and is updated using the generation LLM with one of the following strategies:

- **Generate-new**: instructs the model to create a fresh input from the distilled prompt alone.
- **Mutate-existing**: asks the model to modify the current example (mimicking mutation fuzzers), e.g., swap types, add computations, reorder constructs.
- **Semantic-equiv**: prompts the model to produce an input semantically equivalent to the example but syntactically different (useful for compilers and SMT where form changes trigger new internal paths).

At each iteration, Fuzz4All selects a valid input from the batch as the new example and randomly chooses one of the three strategies, concatenates the distilled prompt + example + strategy text, and queries the generation LLM to produce a new batch of inputs.

Each batch is passed to the SUT and a user-defined oracle detects "interesting" behaviours (e.g., crashes, assertion failures, differential mismatches). The loop continues until the time budget expires, returning the set of detected bugs.

Generation modes: general vs targeted fuzzing

Fuzz4All supports two different modes of execution, based on the original user inputs provided:

- General fuzzing: The distilled prompt summarizes the overall input language (e.g., GCC C docs, CVC5 SMT-LIB overview). The loop explores broadly, maintaining diversity by alternating strategies and continuously refreshing the example.

- Targeted fuzzing: The distilled prompt focuses on a specific feature (e.g., C keywords typedef/union, C++ std::expected, Go atomic/built-in libraries, Java keywords such as instanceof/synchronized/finally). This improves the "hit rate" on the target feature while still discovering off-target behaviors.

Benchmarks and reported results

Across six input languages (C, C++, SMT, Go, Java, Python) and nine SUTs (e.g., GCC/Clang, CVC5/Z3, Go toolchain, OpenJDK javac, Qiskit), Fuzz4All shows consistent advantages over SUT-specific baselines.

Coverage over 24 hours:

Fuzz4All achieves the highest end-of-run coverage on all studied targets, with an average improvement of 36.8% versus the best baseline. Per-target improvements include, +18.8% on g++ (vs. YARPGen), +24.9% on CVC5 (vs. TypeFuzz), +13.7% on Go (vs. go-fuzz), and +60.9% on Java (vs. Hephaestus). Fuzz4All avoids early plateaus by iteratively updating the prompt with examples and alternating strategies, sustaining discovery late into campaigns.

Validity and volume:

Fuzz4All often produces more diverse inputs but with lower average validity than strongly SUT-specific generators. Even with reduced validity, the net number of valid inputs and coverage gains are higher due to LLM breadth and loop diversity; GPU inference is the dominant bottleneck.

Targeted fuzzing effectiveness:

Fuzz4All achieves an average feature hit rate of around 83.0% across targeted campaigns. Targeted prompting yields significant increases in reaching specific features compared to general campaigns, with manageable cross-feature hits (sometimes beneficial for discovery).

Bug finding ability:

98 bugs were reported across studied SUTs, with 64 confirmed by developers as previously unknown. Examples include internal compiler errors in GCC triggered by nuanced feature combinations (e.g., noexcept + std::optional), segmentation faults in Clang, and crashes in Qiskit due to invalid circuit constructs unlikely to be explored by template-based generators.

Limitations mentioned in the paper

- More general prompts for strict languages (e.g., Go with static checks; SMT with tight theory constraints) can yield lower validity, requiring heavier filtering and oracle work. Fuzz4All still improves coverage, but wastes more SUT cycles on invalid inputs.
- The generation LLM (StarCoder in the study) is inference-bound, and fuzzing campaigns are bottlenecked by GPU resources, limiting throughput. Also, the autoprompting overhead (on average $\tilde{3}$ minutes per campaign) is modest, but adds baseline time.
- Autoprompting relies on model priors over recent docs / examples. If documentation is very new, obscure, or under-represented (e.g., emergent quantum APIs), distillation and generation can miss edge cases or "hallucinate" constraints, reducing validity and targeted hit rates.
- Fuzz4All intentionally avoids instrumenting SUTs. While this helps practicality and portability, it limits access to fine-grained internal signals (e.g., coverage edges, value profiling) that grey-box mutation fuzzers exploit to accelerate deep exploration. The method compensates by prompt updates and strategy alternation, but some hard-to-reach behaviours may still benefit from hybridization with feedback-guided loops.

Takeaways and implications for our project

- Fuzz4All's two-LLM pipeline (distillation + generation) enables practical, universal fuzzing across languages and SUTs without bespoke grammars or instrumentation, and makes best use of each LLM's strength / cost trade-off effectively. A multi-LLM approach could be applied in our seed generation project.
- The fuzzing loop is a prompt-conditioned search: distil, generate, validate, then update the prompt with examples while rotating strategies to sustain diversity. This strategy of prompt and input evolution has proven effective, and we can use this as well.
- Possible improvements that Fuzz4All were not able to implement include: pairing prompt evolution with coverage information to further deepen SUT exploration.

2.3.3 LMFuzz

LMFuzz is a universal, code-generating fuzzer that tackles a key weakness of prior LLM-based fuzzers: low validity of generated programs. It introduces two complementary ideas: a multi-armed bandit formulation with Thompson Sampling (TS) to adaptively choose generation operators, and an LLM-driven program-repair loop that fixes compilation/runtime errors on the fly. Together, these components aim to preserve diversity while materially increasing validity and coverage across multiple languages and SUTs [12].

How does LMFuzz work

Prompt selection and scoring:

LMFuzz begins with a small set of prompt candidates P_i - the textual instruction given to the generation LLM that conditions what kind of code to produce for the SUT. For each prompt P_i, the generation LLM produces a batch of 30 programmes (code samples) $x_{i,1}, \ldots, x_{i,30}$. Each programme is then executed against the SUT.

Let program_return$(x_{i,j})$ denote the SUT's exit status when executing programme $x_{i,j}$. The prompt's score is the number of programmes in its batch that run successfully (exit code 0):

$$\text{Score}(P_i) \;=\; \sum_{j=1}^{30} \big(\text{program\_return}(x_{i,j}) = 0\big)$$

LMFuzz selects the prompt with the highest Score(P_i) to seed the fuzzing loop; ties are broken by the prompts' identifier order (smallest identifier wins).

Fuzzing loop:

On each iteration of the fuzzing loop, LMFuzz executes the following steps:

- Generate code with the "generative LLM" (StarCoder-1B in the study) from the current prompt.
- Execute the code; if errors occur, invoke the program-repair loop.
- Select a mutation operator via Thompson Sampling (see below) and mutate the prompt using operator-specific instructions.
- Save generated code and update the prompt for the next iteration.

Generation operators (three "arms"):

- **Generate**: ask the LLM to produce a new program from the current prompt (diversity-first).
- **Mutate**: request a modified version of the previously generated program (fine-grained edits, e.g., inserting functions, tweaking parameters).
- **Semantics**: ask for a semantically equivalent program expressed differently (syntax changes that preserve behaviour), useful for compilers/solvers.

Operator selection via Thompson Sampling

LMFuzz treats operator choice (Generate/Mutate/Semantics) as a multi-armed bandit problem. For each operator, it tracks successes and failures (e.g., "compiled and ran without error" vs. not) and samples a score from a Beta posterior:

Arm k maintains counts α_k (successes) and β_k (failures) and draws

$$\theta_k \sim \text{Beta}(\alpha_k, \beta_k)$$

The operator with maximal sampled θ_k is selected for the next iteration, balancing exploration and exploitation under uncertainty [43]. This probabilistic selection improves over random choice by favouring operators that currently yield valid, high-coverage code while still probing alternatives.

Program-repair loop (LLM-based error correction)

Generated programs frequently include basic issues (missing headers/imports, syntax errors, incorrect API usage). LMFuzz addresses this with a second "correction LLM" (DeepSeek-Coder-6.7B in the study) If the program fails, LMFuzz creates a "code+error" pair by concatenating the original source with compiler/runtime error messages and feeds it to the correction model.

The correction output replaces the program when valid; otherwise, LMFuzz can perform an additional repair cycle (the study evaluates 0/1/2 cycles). One repair cycle typically yields the best trade-off between validity gains and overall coverage (too many repair cycles reduce throughput and program diversity).

Benchmarks and reported results

Coverage versus baselines:

Across GCC, G++, CVC5, Go, and Qiskit, LMFuzz achieved higher or competitive line coverage than both traditional fuzzers (e.g., GrayC, Csmith, TypeFuzz, go-fuzz, MorphQ) and Fuzz4All. Examples from Table 3:

- GCC: LMFuzz 199,595 lines, coverage rate 16.57% (higher than GrayC 13.90% and Csmith 9.27%).
- G++: LMFuzz 223,129 lines, 19.08% (higher than YARPGen 14.25%).
- CVC5: 48,713 lines, 9.12% (higher than TypeFuzz 8.64%).
- Go: 38,024 lines, 12.03% (higher than go-fuzz 11.32%).
- Qiskit: 31,565 lines, 19.63% (slightly higher than Fuzz4All 19.45%, competitive with MorphQ 20.39%).

Validity of generated code

LMFuzz's code validity is roughly twice that of Fuzz4All across five projects (e.g., C/C++ validity in the 60–70% range for LMFuzz vs. 30–37% for Fuzz4All), though still below specialized, traditional generators with strict grammars.

Bug finding ability:

24 bugs reported across five SUTs: GCC (4), Clang (9), CVC5 (2), Qiskit (9); 7 previously unknown confirmed by developers and 17 known/rediscovered (Table 7, Fig. 9).

Ablations:

- Thompson Sampling vs random selection: TS yields higher line coverage across all subjects (Table 4).
- Repair cycles: one cycle improves validity markedly; two cycles further increase validity but reduce coverage/diversity (Table 5).

- Prompt-quality scoring: higher-scored prompts lead to better validity and coverage, validating the initial scoring function.

Limitations mentioned in the paper

- While validity improves substantially over prior LLM fuzzers, it remains below strictly grammar-driven generators. More repair cycles can raise validity but reduce diversity/coverage due to time budget constraints.
- Performance depends on initial prompt quality and model priors; the scoring function helps, but prompts that omit critical headers/imports or feature details degrade validity (Section 4.2.3). Updates to foundation models pose external validity threats.
- The approach is inference-bound; per-iteration generation and repair consume GPU time, limiting the total number of programs explored under fixed budgets.
- Evaluation uses line coverage (via project tools or repository line counts). Without grey-box instrumentation (e.g., edge/branch/value profiling), some deep behaviours may remain under-explored compared to feedback-guided mutation fuzzers.
- The TS bandit selects among three operators. Extending to richer operator families (grammar-aware rewrites, API-sequence planners) may improve generality but complicates posterior credit assignment and sampling dynamics.

Takeaways and implications for our project:

- Modelling operator choice as a bandit with Thompson Sampling gives a principled and effective way to steer LLM generation toward valid, high-coverage programs under uncertainty [12, 43].
- A lightweight, in-loop program-repair mechanism materially boosts validity and practical bug-finding without hand-built grammars.
- For new LLM fuzzers, combining adaptive operator selection, prompt-quality scoring, and on-the-fly repair appears crucial to escape the low-validity trap while retaining diversity; hybridizing with selective feedback signals (e.g., simple coverage counters) could further deepen exploration.

Chapter 3

ElFuzz-Direct

We present ElFuzz-Direct, a modification of ELFuzz [11] that uses the LLM to directly synthesise test inputs, rather than to synthesise the generation-based fuzzers that produce them. ELFuzz solves the hard problem of writing a good input generator by delegating it to an LLM and steering the search with coverage feedback over a so-called *fuzzer space*. Our hypothesis is that the same machinery (LLM-driven mutation, coverage-based strength comparison, and elitist selection) can be applied one level lower, to the inputs themselves. The result is a coverage-guided, LLM-driven seed-corpus synthesiser: it produces a diverse, high-coverage set of seed inputs that can be used as a corpus in their own right or handed to a downstream mutation-based fuzzer such as AFL++ [15, 21].

This chapter describes the design and implementation of ElFuzz-Direct. The development of ElFuzz-Directwas in two major stages. The first stage was a direct replacement of a *generator* with a *collection* of inputs, to most closely preserve the structure of the original pipeline. After discovering issues with this design, the second stage removes collections entirely, and maintains a single evolving population of inputs, as this is a more natural implementation.

3.1 From Generators to Inputs

ELFuzz evolves a population of *generators*: small programs (in the prototype, Python functions) that, when executed, emit a test input for the SUT. Evolution proceeds as an iterative loop in which each generation is mutated by the LLM, the mutants are assessed by the coverage they reach, and the strongest are retained as seeds for the next generation [11]. A crucial property of this design is that the unit of evolution, a generator, is itself a *source* of inputs: a single generator can be executed arbitrarily many times to produce an unbounded number of distinct inputs. Coverage of a generator is therefore measured by running it a fixed number of times and taking the union of the edges its outputs reach, approximating its *cover set* $\mathcal{C}(F)$.

Our project removes the generation step entirely. Instead of asking the LLM to write a program that emits an input, we ask it to emit the input. This eliminates a layer of indirection and the associated cost of executing generators to obtain their outputs. It also sidesteps the need for the LLM to produce syntactically valid *code* in a fixed host language, as it only needs to produce content in the SUT's input format. It also raises an immediate design questionr: in ELFuzz, the unit of evolution is a generator, but a generator is not the same kind of object as an input. A generator stands in for a whole family of inputs, whereas a single input is just one point in the input space. The two designs that follow differ precisely in how they answer the question of what the unit of evolution should become once generators are removed.

Figure 3.1: The ELFuzz evolution loop, which ElFuzz-Direct adapts. Each iteration mutates the current generators with an LLM, places the mutants in the fuzzer space according to their cover sets, and selects a fixed number of elites that maximise the unioned coverage.

3.2 Stage One: Replacing Generators with Collections of Inputs

The most direct translation of ELFuzz would replace each generator with a single input and reuse the rest of the pipeline unchanged. However, this is not an equivalent design. A generator can produce an unbounded number of inputs, and therefore can reach many parts of the SUT. A single input exercises exactly one path through the SUT, so can only ever cover a small, fixed portion of it. Substituting one input for one generator would thus replace a rich object with a far weaker one, and the coverage-driven search would have far less to work with at each point in the population.

To preserve the expressive power of the unit of evolution, our first design replaces each generator with a *collection* of inputs. A collection plays the role that a generator played in ELFuzz: it is the object that is mutated, assessed, and selected. Its cover set is the union of the cover sets of its member inputs, so a collection of many diverse inputs can reach as much of the SUT as a capable generator, while remaining a concrete corpus of seeds rather than a program.

Figure 3.2: Replacing a single generator with a collection of inputs. The collection's cover set is the union of its members' cover sets, so it preserves the role the generator played as the unit of evolution.

3.2.1 Mutating individual inputs

We reuse ELFuzz's three LLM-driven mutation operators – splicing, completion, and infilling – but apply them to individual inputs rather than to generator source code. In ELFuzz these operators act on programs and are realised through fill-in-the-middle (FIM) queries to the model [11]; here they act on the raw bytes of an input, cut at random line boundaries:

- **Completion**: an input is truncated at a random line boundary and the LLM is asked to continue it, extending the input with new, contextually consistent content.
- **Infilling**: two random line boundaries are chosen, removing a middle portion of the input, and the LLM fills the resulting hole via a FIM query while keeping the surrounding content coherent.
- **Splicing**: a prefix taken from one input and a suffix taken from another are joined, and the LLM produces glue content to fuse them into a single valid input. As in ELFuzz, splicing is the operator that combines the strengths of two parents.

These byte-level cuts are the input-space equivalents of ELFuzz's line-level cuts over generator code. Where the original implementation cuts and rejoins lines of a Python program, we cut and rejoin lines of, for example, a JSON document or an SQL script. The mutated input is always assembled from the preserved prefix and suffix plus the LLM-generated bridge, so that even when the model produces poor content the surrounding structure of the parent is retained.

Figure 3.3: The three mutation operators applied to JSON inputs. Shaded regions are taken verbatim from the parent input(s); the remaining region is generated by the LLM to bridge the cut.

3.2.2 Selecting elite collections

Because mutation continually produces new candidates, the population must be pruned to prevent it from growing without bound. ELFuzz achieves this with *max-cover selection*. From the pool it retains a fixed number N of elite generators whose unioned cover set is maximal, approximated greedily [11]. The collection-based design applies exactly the same principle, one level lower. Each collection is scored by its unioned cover set, and the strength pre-order

$$\mathcal{S}(C_i) \sqsubset \mathcal{S}(C_j) \quad \Longleftrightarrow \quad \mathcal{C}(C_i) \subset \mathcal{C}(C_j)$$

ranks collections by the subset relation between their cover sets, just as ELFuzz ranks generators. We retain the N collections whose union of cover sets is largest and discard the rest, keeping the population at a fixed size while favouring elites that collectively expand coverage.

3.2.3 Limitations of the collection-based design

Although this design maps cleanly onto the ELFuzz pipeline, evolving collections rather than generators introduces two problems that do not arise when the unit of evolution is a true generator.

The corpus cannot grow. The population is a fixed number of collections, each holding a fixed number of inputs, so the total number of seeds is bounded throughout the run. With generators this bound is harmless, because each generator can produce an unbounded amount of inputs after evolution finishes. With collections the bound is real; the corpus can never contain more inputs than the initial configuration allows, regardless of how much new coverage the search discovers, and an input can only enter a collection by displacing another.

Elitism discards useful inputs. The collections are disjoint, but coverage is a property of individual inputs, and useful inputs are scattered across collections. Suppose one collection contains an input that uniquely covers some region of the SUT, and a different collection contains an input that uniquely covers another, disjoint region. Max-cover selection chooses whole collections, so if only one of the two collections is retained as an elite, the valuable input in the other is discarded along with it. The unit of selection (a collection) is coarser than the unit that actually carries coverage (an input), so selecting elites loses information and discards genuine progress.

Figure 3.4: Preliminary results for the collection-based design.

3.3 Stage Two: Evolving a Single Population of Inputs

Both limitations above stem from the same root cause: coverage lives on individual inputs, but the collection-based design forces selection and accounting to operate on coarser groups. The second design removes the mismatch by making the input itself the unit of evolution. Rather than many disjoint, fixed-size collections, we maintain a single growing population - a *pool* - of inputs, and every decision is made per input. This is both conceptually simpler and strictly more expressive than the collection-based design.

3.3.1 Initialising the pool

The evolution loop needs an initial pool to mutate. ELFuzz begins its own evolution from a single *generic generator*: a naive seed fuzzer that emits purely random bytes. It relies on LLM-driven evolution to gradually turn that noise into a structured generator. The paper notes that stronger initial templates could improve convergence [11]. Our first design followed the same spirit at the input level: the local LLM was queried to produce the entire starting population from scratch, each input generated independently from a short prompt naming only the target format.

In practice this proved a poor fit for the code-completion models used in the evolution loop. Such models are trained to *continue* existing text or code given substantial surrounding context. When prompted to produce a complete, well-formed input of a given format from a one-line description,

Figure 3.5: From many disjoint collections (left) to a single growing pool (right). Treating the input as the unit of evolution lets every input that contributes coverage survive, and lets the corpus grow without a fixed bound.

they frequently returned empty, truncated, or malformed outputs that the SUT rejected outright and contributed no coverage. A from-scratch prompt gives the model too little to anchor on, and a small local model has weak priors for emitting a valid document of an arbitrary format unprompted.

An intermediate design seeded the pool from a *seed corpus* of real example inputs – small, publicly available samples such as JSON documents, XML files, or SQL scripts – so that the population began as a set of guaranteed in-format, coverage-bearing inputs. Where the local LLM was still used to generate further inputs it was primed with a few corpus examples (few-shot prompting) so it imitated real structure rather than inventing one from nothing. This improved the quality of the initial pool, but introduced a dependency on the availability of a suitable corpus for each target format, and the diversity of the starting population remained limited by whatever the corpus happened to contain.

The final design replaces the real-world corpus with a small set of inputs *pre-generated* by a different, more capable model: Claude Opus 4.8 [**?**]. Before evolution begins, the powerful model is prompted to produce a fixed number of inputs that are each semantically and structurally distinct from one another. This closely mirrors how ELFuzz initialises its own search: just as ELFuzz provides a single Generic generator as the shared base from which all generators evolve, our pre-generated inputs serve as the fixed starting population from which all subsequent mutations depart. The key difference is that our starting population already contains real, diverse instances of the target format rather than a trivially random baseline.

The division of labour between the two models is deliberate. The local code-completion models used in the evolution loop excel at extending and recombining existing text – exactly what the mutation operators require – but are ill-suited to generating varied, semantically interesting documents from scratch, a task that demands a strong prior over the target format and the capacity to produce meaningfully different examples on each call. A large frontier model such as Claude Opus 4.8, prompted to generate k inputs that are each semantically distinct from one another, reliably produces a diverse, well-formed starting population. Because this pre-generation step is performed once per target and its outputs are stored, the cost is paid only at setup time and does not recur across the evolution loop.

3.3.2 The evolution loop

The evolution loop mirrors ELFuzz's three stages, but operates on inputs. The pool begins with the pre-generated inputs described in , after which each generation performs three steps:

1. **Mutation.** A batch of candidate inputs is produced by repeatedly drawing one (or, for splicing, two) inputs at random from the pool and applying one of the three mutation operators

described in .

2. **Coverage assessment.** Each candidate is executed on the SUT and its cover set – the set of edges it reaches – is recorded.

3. **Curation.** Each candidate is compared against the current pool and either added, used to replace an inferior input, or discarded, according to the decision rule below.

3.3.3 Curating the pool

Curation is where the strength pre-order of the fuzzer space is enforced at the level of individual inputs. We say an input with cover set e_1 is *superior* to one with cover set e_2 when it strictly dominates it – that is, when $e_2 \subset e_1$ – which is the input-level instance of ELFuzz's relation $\mathcal{S}(F_i) \sqsubset \mathcal{S}(F_j) \iff \mathcal{C}(F_i) \subset \mathcal{C}(F_j)$. Letting U denote the union of the cover sets of all inputs currently in the pool, each candidate with cover set e is handled as follows:

- If the candidate covers edges not yet in the pool ($e \setminus U \neq \emptyset$), it is added to the pool. Any existing input that the new one strictly dominates is then evicted, since its coverage is now redundant.

- Otherwise the candidate reaches no new edges, but it may still be a better representative of coverage the pool already has. If it strictly dominates some existing input, it replaces the input it dominates by the largest margin.

- Otherwise the candidate is redundant with respect to the pool and is discarded.

A candidate that reaches no edges at all – for example, a malformed input the SUT rejects immediately – is always discarded. We additionally break ties by input size: when a candidate covers exactly the same edges as an existing input but is smaller, it replaces it. This is a deliberate departure from ELFuzz, whose selection rewards coverage with no size penalty; for a *seed corpus*, smaller inputs that achieve the same coverage are preferable, because they mutate and execute more cheaply in a downstream fuzzer.

The effect of this rule is that the pool always retains the union of all coverage discovered so far while continuously pruning inputs whose coverage is subsumed by others. It approximates a Pareto frontier of inputs under the coverage-subset relation, refined by the size penalty – analogous to the max-cover elites of ELFuzz, but maintained incrementally and without a fixed cap on its size.

3.3.4 Properties

This design resolves both limitations of . Because selection now operates on the same unit that carries coverage, *no progress is lost*: two inputs with disjoint coverage both survive, as each contributes edges the other lacks, where under collection-based elitism one of them might have been discarded with its non-elite collection. And because the pool is a single set with no fixed capacity, the corpus is no longer capped by an initial configuration; it admits every input that contributes new coverage.

It is worth being precise about how the pool changes over time, because its size is not monotonic. When a candidate introduces new edges it is added, but it may simultaneously dominate – strictly subsume the cover sets of – several existing inputs at once, all of which are then evicted. The net effect can be a *smaller* pool than before the candidate arrived. Crucially, however, eviction only ever removes an input whose coverage is wholly contained in that of a retained input, so the union of cover sets – the total coverage held by the pool – never decreases. Overall coverage is therefore monotonically non-decreasing across generations, even though the population size is not. The pool tends to *consolidate*: as the search discovers individual inputs that each cover what previously took several, the corpus can become both smaller and stronger, with the size tie-break of reinforcing the same trend. This is exactly the behaviour we want from a seed corpus, where fewer, higher-coverage seeds make for a more efficient downstream fuzzing campaign.

3.4 Implementation

ElFuzz-Direct is implemented as an additional *direct* mode within the existing ELFuzz codebase, reusing its configuration, coverage tooling, and LLM-serving infrastructure wherever possible. The pipeline is driven by two shell scripts that orchestrate a small number of Python components.

Orchestration. An outer script bootstraps the initial pool from the pre-generated inputs () – placing them into the pool, computing their coverage, and retaining those that cover any edges – and then runs the per-generation loop for a configured number of generations or until an optional wall-clock budget is exhausted. Each generation is handled by an inner script that performs the three steps of : mutation, coverage assessment, and curation. To reuse the existing per-generator coverage tooling unchanged, each candidate input is written into its own directory so that the tooling treats it as an independent unit and reports its coverage separately.

Initialisation and mutation. Initialisation is handled by a component that reads the inputs pre-generated by Claude Opus 4.8 and places them into the pool, with each candidate assessed for coverage and discarded if it reaches no edges. Mutants are produced by a second component that draws inputs from the pool and applies the completion, infilling, and splicing operators; the choice of operator per candidate is random over the enabled set, and any operator can be disabled to support ablation studies. To prevent prompts from growing without bound across generations – each generation's outputs become the next generation's parents, so naïvely concatenated prefixes and suffixes would compound – the text fed to the model for each operator is capped at a fixed size, snapped to a line boundary, while the full parent text is still used to assemble the written candidate.

LLM backends. The implementation is model-agnostic and supports two backends. The first serves a local open-weights model through a HuggingFace Text Generation Inference server and uses the model's native fill-in-the-middle tokens for the infilling and splicing operators, reusing the same FIM machinery as ELFuzz's generator mutation. The second targets a hosted API model and instead drives the operators with natural-language instructions, post-processing the response to strip markdown fences and other extraneous formatting. Supporting both lets us compare a small local model against a larger hosted one, as set out in the project goals.

Curation. The decision rule of is implemented by a curator that ingests the per-candidate coverage map and the current pool index, applies the add/replace/discard rule to each candidate, and writes the updated pool together with a record of every candidate's fate. The pool is stored as a flat directory of inputs alongside an index mapping each input to its cover set and size, so that the union U and the domination checks can be evaluated efficiently each generation.

Chapter 4

Evaluation

The aim of this project is to test a single hypothesis. ELFuzz produces seed inputs *indirectly*: it uses an LLM to synthesise generation-based fuzzers, and those fuzzers are then run to emit the inputs [11]. ElFuzz-Direct removes the intermediate program and asks the LLM for the inputs directly. Our conjecture is that this is a **more direct route to diverse, semantically interesting inputs**, and therefore to **more varied coverage of the system under test (SUT)** – which is ultimately what a fuzzing campaign is trying to achieve. By eliminating the generator, the model's knowledge of the input format is spent on the inputs themselves rather than on writing a program that happens to produce them, and coverage feedback selects directly over the objects we care about.

This chapter sets out how we test that hypothesis. The whole evaluation is framed as a like-for-like comparison between `elfuzz synth` (the original, generator-synthesising pipeline) and `elfuzz direct` (our input-generating modification), run under identical models, hardware, and time budgets. We describe the research questions (Section 4.1), the experimental setup and protocol (Section 4.2), and the metrics and result tables/figures that will carry the comparison (Section 4.3). Quantitative analysis and discussion are deferred until the full set of runs has completed.

4.1 Aims and Research Questions

We restate the central hypothesis as the lens through which every result is read:

> *Generating inputs directly with an LLM yields a more diverse, more semantically interesting seed corpus – and hence more varied SUT coverage – than synthesising fuzzers that generate those inputs.*

We decompose this into three research questions. The numbering mirrors ELFuzz's own evaluation, where RQ1 concerns input-generation effectiveness and RQ2 concerns downstream bug-finding [11].

- **RQ1 (seed quality).** How much SUT coverage does each tool's seed corpus achieve *on its own*, before any mutation fuzzing? This measures the raw quality of what each approach produces, and is the most direct test of the hypothesis: if directly generated inputs are more varied, they should cover more of the SUT.
- **RQ2 (downstream fuzzing).** When the seed corpus is used to bootstrap a grey-box mutation fuzzer (AFL++ [15, 21]), how does coverage grow over the campaign, and where does it end? A more diverse corpus should let the fuzzer past shallow checks sooner and reach a higher plateau.
- **RQ3 (search behaviour).** How does coverage evolve *during* synthesis versus during pool evolution, and how does the corpus size behave – in particular, does ElFuzz-Direct exhibit the consolidation behaviour predicted in Section **??**, where the pool can shrink while coverage only increases?

4.2 Experimental Setup

4.2.1 Systems under test

We evaluate on a subset of the seven real-world SUTs used by ELFuzz, spanning structured data formats and language interpreters [11]. Table 4.1 lists them with their input format and approximate size; the SUTs exercise a range of input complexities, from a JSON parser to an SMT solver.

SUT	Input format	Approx. size (LoC)
jsoncpp	JSON	25 k

Table 4.1: Systems under test evaluated in this project (a subset of the seven ELFuzz SUTs). Sizes are approximate.

4.2.2 Configurations compared

The two configurations are `elfuzz synth` and `elfuzz direct`. To isolate the effect of the design difference, every other factor is held fixed: both use the same code-completion LLM (Qwen2.5-Coder-1.5B, the small local model chosen to fit the available GPU), run on the same single-workstation host, and are given identical time budgets. Both pipelines end by feeding their seed corpus to the same AFL++ configuration, so any downstream difference is attributable to the seeds rather than to the fuzzer.

4.2.3 Protocol

Each SUT is run end-to-end through the same four-stage pipeline for both tools:

1. **Synthesis / evolution (6 h).** `synth` evolves a population of generators; `direct` evolves the input pool of Chapter **??** (ElFuzz-Direct). Both are given a 6-hour wall-clock budget.
2. **Corpus production.** For `synth`, a separate `produce` step runs the evolved elite fuzzers for **200 seconds in total**, which is already sufficient to emit well over one million candidate inputs. `direct` has no `produce` step: its evolved pool *is* the corpus. Both corpora are then minimised to remove inputs that are redundant for coverage.
3. **Seed-coverage benchmark (RQ1).** The minimised corpus is executed against the SUT and the total coverage it reaches is recorded.
4. **AFL++ campaign (6 h, RQ2).** The minimised corpus seeds a 6-hour AFL++ campaign (one repetition in the current runs), and coverage is logged over time.

Instrumentation caveat. The coverage counts produced during synthesis/evolution and the `edges_found` reported by AFL++ come from different instrumentation, so the two phases are *not* directly comparable in absolute terms. We therefore only ever compare `synth` against `direct` *within* a single metric, never across the synthesis and AFL phases.

4.2.4 Metrics

- **Seed coverage** – edges covered by the minimised seed corpus alone (RQ1).
- **AFL coverage over time** – `edges_found` as a function of wall-clock time, and the final value at 6 h (RQ2).
- **Bitmap coverage** – AFL's percentage of the coverage bitmap exercised, at end of campaign.
- **Throughput** – total executions and executions per second, as a sanity check that campaigns are comparable.
- **Corpus size** – number of seeds, and (for `direct`) the pool size across generations, to evidence the consolidation behaviour (RQ3).

- **Evolution coverage** – union of edges covered by the whole population/pool per generation, during synthesis (RQ3).
- **Crashes and hangs** – saved crashing and hanging inputs found by the campaign.

4.3 Results

This section presents the result tables and figures. Numbers are filled in as each SUT's runs complete; analysis and comparison are deferred until the full set is available.

4.3.1 Seed coverage (RQ1)

Table 4.2 reports the coverage achieved by each tool's minimised seed corpus on its own. This is the most direct measurement of seed quality.

SUT	synth	direct
jsoncpp	568	463

Table 4.2: Seed-corpus coverage (edges covered by the minimised corpus, before any mutation fuzzing) for each tool. Higher is better. Values are pending for runs not yet completed.

4.3.2 Coverage during evolution (RQ3)

Figure 4.1 shows how the coverage of the whole population (for `synth`) or pool (for `direct`) grows over the generations of the 6-hour synthesis phase. These counts use the synthesis-phase instrumentation and so are comparable between the two tools, but not against the AFL figures below.

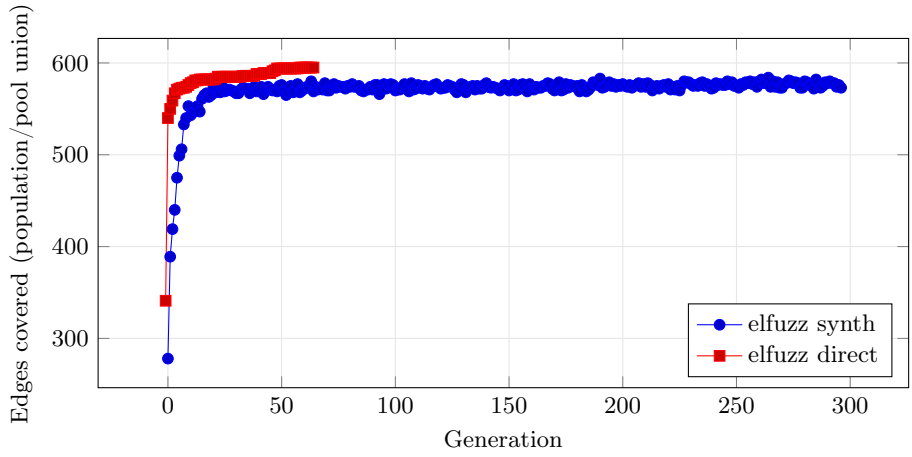

Figure 4.1: Coverage over the synthesis/evolution phase on jsoncpp: union of edges covered by the whole population (synth) or pool (direct), per generation.

4.3.3 Downstream AFL++ coverage (RQ2)

Figure 4.2 shows AFL++ edge coverage over the 6-hour campaign for each tool's seed corpus, and Table 4.3 summarises the end-of-campaign statistics.

4.4 Threats to Validity

We note the following limitations, to be expanded once results are in. The AFL++ campaigns currently use a **single repetition** per configuration, so per-run variance from the stochasticity of fuzzing is not yet quantified; multiple repetitions are planned where the time budget allows. Both

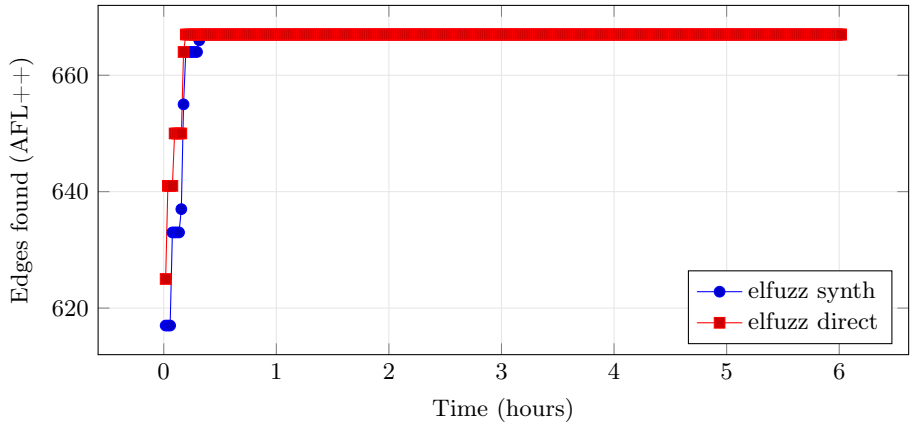

Figure 4.2: AFL++ coverage over time on jsoncpp, seeded with each tool's minimised corpus. Both campaigns run for 6 hours under identical settings.

SUT	Tool	Edges	Bitmap	Corpus	Execs	Crashes
jsoncpp	synth	667	12.74%	1311	166.9 M	0
	direct	667	12.74%	1223	156.6 M	0

Table 4.3: End-of-campaign AFL++ statistics (6 h, one repetition): edges found, bitmap coverage, final corpus size, total executions, and saved crashes. Values are pending for runs not yet completed.

tools use a **small local model**, which bounds the quality of generation for both; a larger model could change the absolute numbers, though the relative comparison should hold. The evaluation covers a **subset of SUTs** and so may not generalise to all input formats. Finally, as noted in Section 4.2.3, synthesis-phase and AFL-phase coverage use different instrumentation and must not be compared across phases.

Appendix A

First Appendix

Bibliography

[1] GlobeNewswire. CISQ publishes the cost of poor software quality in the US: A 2022 report; 2022. Available from: https://www.globenewswire.com/news-release/2022/12/06/2568427/0/en/CISQ-Publishes-the-Cost-of-Poor-Software-Quality-in-the-US-A-2022-Report.html.

[2] Inc S. Samet L, editor. Software quality issues in the U.S. cost an estimated $2.41 trillion in 2022; 2022. Available from: https://www.prnewswire.com/news-releases/software-quality-issues-in-the-us-cost-an-estimated-2-41-trillion-in-2022--301695684.html.

[3] Manès VJM, Han H, Han C, Cha SK, Egele M, Schwartz EJ, et al. The art, science, and engineering of fuzzing: A survey. IEEE Transactions on Software Engineering. 2021;47(11):2312-31. Available from: https://doi.org/10.1109/TSE.2019.2946563.

[4] Yu ea. Fuzzing: Progress, challenges, and perspectives. Computers, Materials & Continua. 2024;78(1):1–29. Available from: https://doi.org/10.32604/cmc.2023.042361.

[5] Blog GS. AI-Powered fuzzing: Breaking the bug hunting barrier; 2023. Available from: https://security.googleblog.com/2023/08/ai-powered-fuzzing-breaking-bug-hunting.html.

[6] Blog GS. Leveling up fuzzing: Finding more vulnerabilities with AI; 2024. Available from: https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html.

[7] Wang Y, Jia P, Liu L, Huang C, Liu Z. A systematic review of fuzzing based on machine learning techniques. PLOS ONE. 2020 08;15(8). Available from: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0237749.

[8] Blog GS. Scaling security with AI: from detection to solution; 2024. Available from: https://security.googleblog.com/2024/01/scaling-security-with-ai-from-detection.html.

[9] Xia CS, Paltenghi M, Tian JL, Pradel M, Zhang L. Fuzz4All: Universal fuzzing via large language models; 2023. Available from: https://arxiv.org/abs/2308.04748.

[10] Yang C, Deng Y, Lu R, Yao J, Liu J, Jabbarvand R. WhiteFox: White-box compiler fuzzing empowered by large language models; 2023. Available from: https://arxiv.org/abs/2310.15991.

[11] Chen C, Dolan-Gavitt B, Lin Z. ELFuzz: Efficient input generation via LLM-driven synthesis over fuzzer space; 2025. Available from: https://arxiv.org/abs/2506.10323.

[12] Lin R, Wang R, Hu G, Xu X. LMFuzz: Program repair fuzzing based on large language models. Automated Software Engineering. 2025 10;33. Available from: https://doi.org/10.1007/s10515-025-00568-8.

[13] Miller BP, Fredriksen L, So B. An empirical study of the reliability of UNIX utilities. University of Wisconsin–Madison; 1990. Available from: https://pages.cs.wisc.edu/~bart/fuzz/.

[14] Miller BP, Cooksey G, Moore F. Fuzz revisited: A re-examination of the reliability of UNIX utilities and services. University of Wisconsin–Madison; 2006. Available from: https://pages.cs.wisc.edu/~bart/fuzz/.

[15] Böhme M, Pham VT, Roychoudhury A. Coverage-based greybox fuzzing as markov chain. In: CCS'16: 2016 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery; 2016. p. 1032–1043. Available from: https://doi.org/10.1145/2976749.2978428.

[16] Serebryany K, Bruening D, Potapenko A. AddressSanitizer: A fast address sanity checker. In: 2012 USENIX Annual Technical Conference. USENIX; 2012. p. 309–318. Available from: https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany.

[17] LLVM Project. MemorySanitizer; 2015. Available from: https://clang.llvm.org/docs/MemorySanitizer.html.

[18] LLVM Project. UndefinedBehaviorSanitizer; 2015. Available from: https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html.

[19] Serebryany K, Iskhodzhanov T. ThreadSanitizer: Data race detection in practice. In: Workshop on Binary Instrumentation and Applications (WBIA'09); 2009. p. 62–71. Available from: https://doi.org/10.1145/1791194.1791203.

[20] Le V, Afshari M, Su Z. Compiler validation via equivalence modulo inputs. SIGPLAN Not. 2014;49:216–226. Available from: https://doi.org/10.1145/2594291.2594334.

[21] Serebryany K. libFuzzer: A library for coverage-guided fuzz testing; 2016. Available from: https://llvm.org/docs/LibFuzzer.html.

[22] Yang X, Chen Y, Eide E, Regehr J. Finding and understanding bugs in C compilers. SIGPLAN Not. 2011 06;46(6):283–294. Available from: https://doi.org/10.1145/1993316.1993532.

[23] Aschermann C, Schumilo S, Gawlik R, Schinzel S, Holz T. NAUTILUS: Fishing for deep bugs with grammars. In: Network and Distributed Systems Security (NDSS) Symposium 2019. NDSS; 2019. Available from: https://dx.doi.org/10.14722/ndss.2019.23412.

[24] Holler C, Herzig K, Zeller A. Fuzzing with code fragments. In: 21st USENIX Security Symposium. USENIX Association; 2012. p. 445–458. Available from: https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/holler.

[25] Bellard F. QEMU, a fast and portable dynamic translator. In: FREENIX Track: 2005 USENIX Annual Technical Conference; 2005. p. 41–46. Available from: https://www.usenix.org/legacy/event/usenix05/tech/freenix/bellard.html.

[26] Luk CK, Cohn R, Muth R, Patil H, Klauser A, Lowney G, et al. Pin: building customized program analysis tools with dynamic instrumentation. SIGPLAN Not. 2005 06;40(6):190–200. Available from: https://doi.org/10.1145/1064978.1065034.

[27] Godefroid P, Klarlund N, Sen K. DART: Directed automated random testing. SIGPLAN Not. 2005;40:213–223. Available from: https://doi.org/10.1145/1065010.1065036.

[28] Cadar C, Dunbar D, Engler D. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In: 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI'8). USENIX; 2008. p. 209–224. Available from: https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf.

[29] Godefroid P, Levin MY, Molnar DA. SAGE: Whitebox fuzzing for security testing. Communications of the ACM. 2012;55(3):40–44. Available from: https://doi.org/10.1145/2093548.2093564.

[30] GNU Project. gcov: a test coverage program; 2024. Available from: https://gcc.gnu.org/onlinedocs/gcc/Gcov.html.

[31] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al.. Attention Is All You Need; 2017. Available from: https://arxiv.org/abs/1706.03762.

[32] Devlin J, Chang MW, Lee K, Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding; 2018. Available from: https://arxiv.org/abs/1810.04805.

[33] Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, et al.. Language models are few-shot learners; 2020. Available from: https://arxiv.org/abs/2005.14165.

[34] Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, et al.. PaLM: Scaling language modeling with pathways; 2022. Available from: https://arxiv.org/abs/2204.02311.

[35] Ouyang L, Wu J, Jiang X, Almeida D, Wainwright CL, Mishkin P, et al.. Training language models to follow instructions with human feedback; 2022. Available from: https://arxiv.org/abs/2203.02155.

[36] Touvron H, Lavril T, Izacard G, Martinet X, Lachaux MA, Lacroix T, et al.. LLaMA: Open and efficient foundation language models; 2023. Available from: https://arxiv.org/abs/2302.13971.

[37] Li R, Allal B, Zi Y, Muennighoff N, Kocetkov D, Mou C, et al.. StarCoder: may the source be with you!; 2023. Available from: https://arxiv.org/abs/2305.06161.

[38] Rozière B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, et al.. Code llama: Open foundation models for code; 2024. Available from: https://arxiv.org/abs/2308.12950.

[39] Chen J, Patra J, Pradel M, Xiong Y, Zhang H, Hao D, et al. A survey of compiler testing. ACM Comput Surv. 2020 02;53(1). Available from: https://doi.org/10.1145/3363562.

[40] Lyu Y, Xie Y, Chen P, Chen H. Prompt fuzzing for fuzz driver generation; 2024. Available from: https://arxiv.org/abs/2312.17677.

[41] 2023 IEEE Symposium on Security and Privacy (SP). UTopia: Automatic generation of fuzz driver using unit tests; 2023. Available from: https://ieeexplore.ieee.org/abstract/document/10179394.

[42] Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. KernelGPT: Enhanced kernel fuzzing via large language models. ACM; 2025. Available from: http://dx.doi.org/10.1145/3676641.3716022.

[43] Thompson WR. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. Biometrika. 1933 12;25:285. Available from: https://doi.org/10.2307/2332286.